



Tensor Decomposition

Authors: Hermitian Operator & Sakura

Санкт-Петербург, 2026

ПРЕДИСЛОВИЕ

“The darker the night, the brighter the stars.”

“Even the darkest night will end and the sun will rise.”

Всех рады приветствовать в новом семестре! В конце прошлого мы начали разбираться с форматами хранения матричных данных. Однако в нейронных сетях, помимо матриц, активно используются тензоры. Они не просто подвержены тем же проблемам, что и матрицы, но и обладают своим специфическим вызовом — “проклятием размерности”. Чтобы научиться преодолевать эти трудности, мы познакомимся с инструментом тензорных разложений.

Особое внимание мы уделим ТТ-разложению (Tensor Train). Его главная сила в том, что оно позволяет перейти от экспоненциального роста требуемой памяти к линейному (относительно количества осей тензора). Это станет прочным фундаментом для понимания того, как оптимизировать тяжеловесные нейронные сети, превращая их в тензоризованные модели, которые в десятки раз компактнее оригиналов, но не уступают им в точности.

А чтобы узнать, как эта “магия”, именуемая прикладной линейной алгеброй, возможна на практике — приступайте к выполнению. Удачи!

ПРАВИЛА ВЫПОЛНЕНИЯ РАБОТЫ

1. **Запрещено использовать сторонние библиотеки**, не входящие в стандартную поставку Python¹. Разрешены только стандартные модули Python.
2. **Все численные методы должны быть реализованы самостоятельно.**
3. **Запрещено использовать искусственный интеллект** или готовые инструменты генерации кода для получения полного решения. Искусственный интеллект допускается только для изучения примеров, генерации подсказок или комментариев, но не для подмены собственной реализации.

¹Список стандартных библиотек: <https://docs.python.org/3/library/>

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Для начала вспомним, что такое тензор и некоторые операции над ним.

Под d -мерным тензором будем понимать многомерный массив:

$$\mathcal{A} = \{a_{i_1 \dots i_d}\}_{i_1, \dots, i_d=1}^{n_1, \dots, n_d} \in \mathbb{R}^{n_1 \times \dots \times n_d}.$$

Проблема: количество хранимых чисел с ростом d увеличивается экспоненциально, это явление называется *проклятием размерности*.

Внешним^a (тензорным) произведением $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$, $\mathcal{B} \in \mathbb{R}^{m_1 \times \dots \times m_D}$ назовем $(\mathcal{A} \circ \mathcal{B}) \in \mathbb{R}^{n_1 \times \dots \times n_d \times m_1 \times \dots \times m_D}$:

$$(\mathcal{A} \circ \mathcal{B})_{i_1 \dots i_d j_1 \dots j_D} = a_{i_1 \dots i_d} b_{j_1 \dots j_D}.$$

^aЗдесь речь идет об outer product, а не об exterior product.

Тензор $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$ называется **тензором ранга^a 1**, если он представим в виде внешнего произведения d векторов:

$$\mathcal{A} = a^{(1)} \circ a^{(2)} \circ \dots \circ a^{(d)},$$

то есть покомпонентно

$$a_{i_1 \dots i_d} = a_{i_1}^{(1)} a_{i_2}^{(2)} \dots a_{i_d}^{(d)}.$$

^aНе путать с порядком тензора, здесь имеется в виду декомпозиционный ранг

Можем ли мы хранить тензор в виде набора векторов $\{a^{(i)}\}_{i=1}^d$, чтобы бороться с проклятием размерности? Оказывается, что можем, для хранения тензора в его явном виде нам нужно хранить $\prod_{k=1}^d n_k$ чисел, а в случае набора векторов мы храним лишь $\sum_{k=1}^d n_k$.

Конечно, существуют тензоры высших рангов, о них поговорим несколько позже.

Изменением формы^a тензора $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$ называется операция перегруппировки его элементов в новый тензор

$$\mathcal{B} \in \mathbb{R}^{m_1 \times \dots \times m_p},$$

выполняемая при строгом условии сохранения общего числа элементов:

$$\prod_{i=1}^d n_i = \prod_{j=1}^p m_j.$$

Логически эта операция эквивалентна вытягиванию исходного тензора в плоский одномерный вектор и его последующей «упаковке» в новую заданную форму.

НВ: точный порядок обхода элементов при вытягивании и заполнении определяется соглашением.

^a Англ. reshape

В математике и программных реализациях по умолчанию используется *лексикографический* порядок (row-major или же C-order), при котором быстрее всего меняется самый последний индекс. **Обратите внимание**, что при выполнении практической части лабораторной работы следует использовать именно его.

Частным случаем операции reshape является развёртка тензора.

Развёрткой^a тензора $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$ по моде k называется матрица

$$A_{(k)} \in \mathbb{R}^{n_k \times (n_1 \dots n_{k-1} n_{k+1} \dots n_d)},$$

полученная из тензора перегруппировкой его элементов так, что индекс i_k становится индексом строки, а все остальные индексы объединяются в индекс столбца.

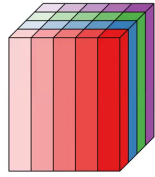
НВ: точный порядок столбцов определяется соглашением.

^a Англ. unfolding, matricization

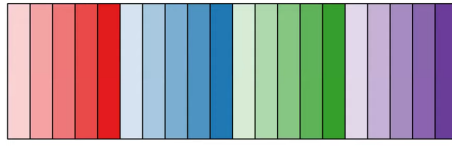
На рисунках² ниже представлены примеры разверток.

²Изображения взяты из видеоролика: https://www.youtube.com/watch?v=V_ugq2iIjxE.

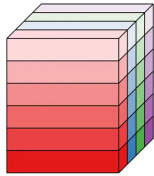
$$\mathcal{X} : m \times n \times p$$



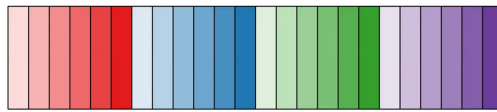
$$\mathbf{X}_{(1)} : m \times np$$



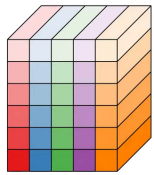
$$\mathcal{X} : m \times n \times p$$



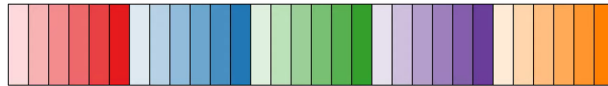
$$\mathbf{X}_{(2)} : n \times mp$$



$$\mathcal{X} : m \times n \times p$$



$$\mathbf{X}_{(3)} : p \times mn$$



Пусть $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$, $U \in \mathbb{R}^{m \times n_k}$. Произведением тензора $\mathcal{A}$ на матрицу U по моде k называется тензор

$$\mathcal{B} = \mathcal{A} \times_k U \in \mathbb{R}^{n_1 \times \dots \times n_{k-1} \times m \times n_{k+1} \times \dots \times n_d},$$

элементы которого определяются формулой

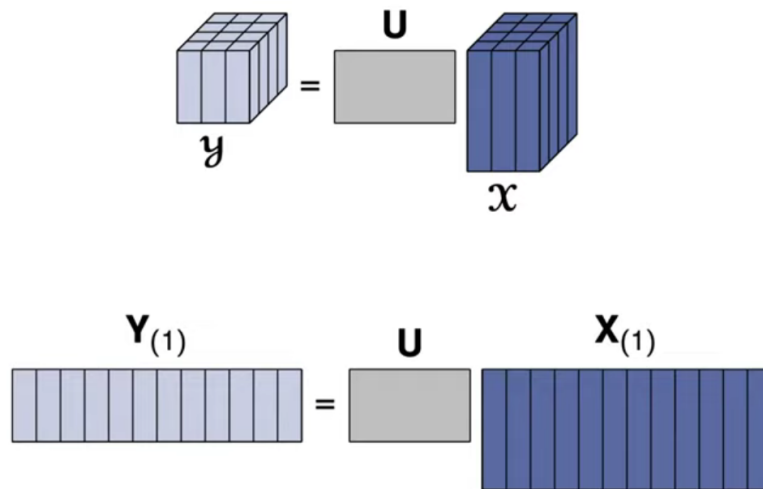
$$b_{i_1 \dots i_{k-1} j i_{k+1} \dots i_d} = \sum_{i_k=1}^{n_k} u_{ji_k} a_{i_1 \dots i_d}.$$

NB: в терминах развёрток это эквивалентно матричному соотношению

$$\mathcal{B}_{(k)} = U A_{(k)}.$$

NB: произведение тензора по моде k показывает, как тензор преобразуется при замене базиса в k -ой моде.

При вычислении произведения $\mathcal{Y} = \mathcal{X} \times_1 U$ получится следующая картина³:



Вернемся к нашей идее разложить тензор на компоненты, чтобы хранить меньше элементов. Для начала посмотрим на примере матричного разложения.

³Изображение взято из видеоролика: https://www.youtube.com/watch?v=V_ugq2iIjxE.

Пусть $A \in \mathbb{R}^{m \times n}$. **Сингулярным разложением матрицы^a** A называется представление

$$A = U \Sigma V^T,$$

где:

- $U \in \mathbb{R}^{m \times m}$ — ортогональная матрица;
- $V \in \mathbb{R}^{n \times n}$ — ортогональная матрица;
- $\Sigma \in \mathbb{R}^{m \times n}$ — диагональная^b матрица с неотрицательными элементами $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ на диагонали.

NB: векторы, составляющие матрицы U , V — *сингулярные векторы*, а диагональные элементы Σ — *сингулярные числа*.

^aАнгл. singular value decomposition (SVD)

^b $\sigma_{ij} = 0$ при $i \neq j$

Можно сказать, что большее значение сингулярного числа соответствует большему “вкладу” соответствующей пары сингулярных векторов в значение матрицы A .

При малых значениях σ_i имеет смысл отбросить их, чтобы хранить приближенное значение матрицы A с меньшими затратами памяти.

Пусть r — номер сингулярного числа, после которого все остальные отбрасываются. Тогда доля объясненной дисперсии (“сохраненной информации”) вычисляется как:

$$\frac{\sum_{i=1}^r \sigma_i^2}{\sum_{i=1}^{\min(m,n)} \sigma_i^2} \geq \alpha,$$

где $\alpha \in [0; 1]$.

Усечённым SVD ранга r называется приближение

$$A \approx A_r = U_r \Sigma_r V_r^T,$$

где оставлены только первые r сингулярных чисел и соответствующие им сингулярные векторы.

Теорема (Эккарта–Янга): Среди всех матриц ранга^a не выше r матрица A_r является наилучшим приближением к A в норме Фробениуса^b:

$$\arg \min_{B: \text{rank}(B) \leq r} \|A - B\|_F = U_r \Sigma_r V_r^T = A_r.$$

^aДля матриц декомпозиционный ранг эквивалентен минорному. Декомпозиционный ранг матрицы — минимальное количество слагаемых вида uv^T , дающее в сумме эту матрицу.

^bЧастный случай p -нормы для $p = 2$:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}.$$

В терминах усеченных матриц полученную ранее оценку объясненной дисперсии в норме Фробениуса можно переписать:

$$\frac{\|A_r\|_F^2}{\|A\|_F^2} = \frac{\sum_{i=1}^r \sigma_i^2}{\sum_{i=1}^{\min(m,n)} \sigma_i^2} \geq \alpha.$$

Заметим, что SVD можно рассматривать как разложение матрицы в сумму тензоров ранга 1:

$$A = \sum_{k=1}^r \sigma_k u_k \circ v_k.$$

Можно ли обобщить идею SVD разложения на тензоры? Оказывается, что можно.

Каноническим разложением^a тензора

$$\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$$

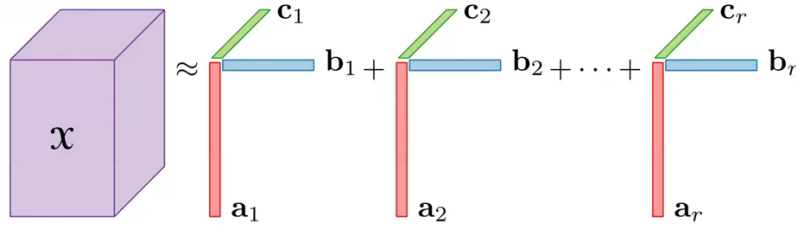
называется представление его в виде суммы тензоров ранга 1:

$$\mathcal{A} = \sum_{r=1}^R a_r^{(1)} \circ a_r^{(2)} \circ \dots \circ a_r^{(d)}.$$

^aЕго также называют CP-разложением, аббревиатура происходит от двух названий, предложенных разными группами ученых: CANDECOMP (Canonical Decomposition), PARAFAC (Parallel Factor Analysis).

Например⁴, для $\mathcal{X} \in \mathbb{R}^{n_1 \times n_2 \times n_3}$:

⁴Изображение взято из видеоролика: https://www.youtube.com/watch?v=V_ugq2i1jxE.



Таким образом, SVD является частным случаем CP-разложения для тензоров порядка 2.

Существует несколько способов записи канонического разложения:

- через внешние произведения:

$$\mathcal{A} = \sum_{r=1}^R a_r \circ b_r \circ c_r;$$

- покомпонентно:

$$a_{ijk} = \sum_{r=1}^R a_{ir} b_{jr} c_{kr};$$

- через матрицы факторов:

Если

$$A = [a_1, \dots, a_R] \in \mathbb{R}^{n_1 \times R}, \quad B = [b_1, \dots, b_R] \in \mathbb{R}^{n_2 \times R}, \quad C = [c_1, \dots, c_R] \in \mathbb{R}^{n_3 \times R},$$

то разложение обозначают кратко:

$$\mathcal{A} = \llbracket A, B, C \rrbracket.$$

Ранее мы определили, какие тензоры имеют ранг 1. Обобщим определение ранга.

Рангом тензора $\mathcal{A}$ называется минимальное число R , для которого существует точное CP-разложение:

$$\text{rank}(\mathcal{A}) = \min \left\{ R \mid \mathcal{A} = \sum_{r=1}^R a_r^{(1)} \circ \dots \circ a_r^{(d)} \right\}.$$

NB: для матриц это определение совпадает с минорным рангом матрицы.

Можно задаться вопросом: а обладает ли каноническое разложение свойством единственности? Оказывается, что при некоторых условиях – да.

Пусть $A \in \mathbb{R}^{n \times m}$ — матрица. **Крускаловским рангом** матрицы A называется максимальное число k , такое что любые k столбцов матрицы A линейно независимы.

Обозначение:

$$\text{krank}(A).$$

NB: всегда выполняется

$$\text{krank}(A) \leq \text{rank}(A).$$

Теорема (Крускала): Пусть тензор порядка N имеет CP-разложение

$$\mathcal{X} = \llbracket A^{(1)}, \dots, A^{(N)} \rrbracket$$

ранга R , и выполнено условие^a

$$\sum_{n=1}^N \text{krank}(A^{(n)}) \geq 2R + (N - 1).$$

Тогда это разложение единственно с точностью до:

- перестановки слагаемых;
- перенормировки факторов.

^aСм. оригинальное доказательство в работе J. B. Kruskal, *Three-way arrays: rank and uniqueness of trilinear decompositions*, Linear Algebra and its Applications, 1977. DOI: 10.1016/0024-3795(77)90069-6.

Для нахождения SVD разложения существует точный и устойчивый алгоритм, однако с CP-разложением возникают проблемы:

- задача вычисления точного ранга тензора является NP-трудной⁵;
- задача нахождения наилучшего приближения заданного ранга может не иметь решения;
- вычисление CP-разложения обычно проводится итерационными методами и не гарантирует нахождение глобального минимума.

Каноническое разложение не единственный способ разложить тензор на компоненты, посмотрим на другие варианты.

⁵Подробнее о классах P и NP в видеоролике: <https://youtu.be/D77FLj-nZ5M>.

Разложением Такера тензора

$$\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$$

называется представление

$$\mathcal{A} = \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \dots \times_d U^{(d)},$$

где

- $\mathcal{G} \in \mathbb{R}^{r_1 \times \dots \times r_d}$ — ядро;
- $U^{(k)} \in \mathbb{R}^{n_k \times r_k}$ — факторные матрицы.

NB: CP-разложение — частный случай разложения Такера.

Существует несколько способов записи разложения Такера:

- через внешние произведения:

$$\mathcal{A} = \sum_{\alpha_1=1}^{r_1} \dots \sum_{\alpha_d=1}^{r_d} g_{\alpha_1 \dots \alpha_d} u_{\alpha_1}^{(1)} \circ \dots \circ u_{\alpha_d}^{(d)};$$

- покомпонентно:

$$a_{i_1 \dots i_d} = \sum_{\alpha_1=1}^{r_1} \dots \sum_{\alpha_d=1}^{r_d} g_{\alpha_1 \dots \alpha_d} u_{i_1 \alpha_1}^{(1)} \dots u_{i_d \alpha_d}^{(d)};$$

- через матрицы факторов и ядро:

$$\mathcal{A} = \llbracket \mathcal{G}; U^{(1)}, \dots, U^{(d)} \rrbracket.$$

Разложение Такера тоже страдает от проклятия размерности: количество элементов в ядре $\mathcal{G}$ с ростом d увеличивается экспоненциально. Хочется найти такое разложение, которое не страдает от проклятия размерности и нетрудно находится численно.

Тензор $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$ представлен в формате **Tensor Train**^a, если каждый его элемент можно записать в виде

$$a_{i_1 \dots i_d} = G_1[i_1] \cdot G_2[i_2] \cdots G_d[i_d],$$

где

- $G_k \in \mathbb{R}^{r_{k-1} \times n_k \times r_k}$ – ТТ-ядро;
- при фиксированном i_k получаем матрицу

$$G_k[i_k] \in \mathbb{R}^{r_{k-1} \times r_k};$$

- числа $r_1, \dots, r_{d-1}$ называются ТТ-рангами, причем

$$r_0 = r_d = 1.$$

^aНазвание пошло от идеи, что тензор представляется в виде последовательных ”вагончиков” – ядер G_k .

Покомпонентная запись тензора в ТТ-формате выглядит следующим образом:

$$a_{i_1 \dots i_d} = \sum_{\alpha_1=1}^{r_1} \cdots \sum_{\alpha_{d-1}=1}^{r_{d-1}} G_1(1, i_1, \alpha_1) G_2(\alpha_1, i_2, \alpha_2) \cdots G_d(\alpha_{d-1}, i_d, 1).$$

Для наглядности на рисунке 1 представлен пример вычисления компоненты тензора порядка 4 по его ТТ-разложению.

$$\mathcal{A}_{2423} = G_1 \times G_2 \times G_3 \times G_4$$

$i_1 = 2$ $i_2 = 4$ $i_3 = 2$ $i_4 = 3$

Рис. 1: Пример вычисления компоненты тензора порядка 4 по ТТ-разложению. Источник: презентация “Tensor Train Decomposition in Machine Learning”, доступно по ссылке: <https://www.slideshare.net/slideshow/tensor-train-decomposition-in-machine-learning/67084738>.

Число параметров в ТТ-формате равно

$$\sum_{k=1}^d r_{k-1} n_k r_k.$$

Если все $n_k = n$, все внутренние ранги равны r , то это порядка dnr^2 , то есть линейно по d . Мы успешно победили проклятие размерности.

Особенностью ТТ-разложения также является возможность производить операции над тензорами в ТТ-формате.

Поэлементная сумма тензоров.

Пусть тензоры $\mathcal{A}$, $\mathcal{B}$ заданы в ТТ-формате:

$$\mathcal{A}_{i_1 \dots i_d} = G_1^{\mathcal{A}}[i_1] \cdots G_d^{\mathcal{A}}[i_d], \quad \mathcal{B}_{i_1 \dots i_d} = G_1^{\mathcal{B}}[i_1] \cdots G_d^{\mathcal{B}}[i_d].$$

Тогда, чтобы найти ТТ-формат тензора

$$\mathcal{C} = \mathcal{A} + \mathcal{B}, \quad \mathcal{C}_{i_1 \dots i_d} = \mathcal{A}_{i_1 \dots i_d} + \mathcal{B}_{i_1 \dots i_d},$$

необходимо построить его ТТ-ядра как:

$$G_k^{\mathcal{C}}[i_k] = \begin{bmatrix} G_k^{\mathcal{A}}[i_k] & 0 \\ 0 & G_k^{\mathcal{B}}[i_k] \end{bmatrix}, \quad k = 2, \dots, d-1,$$

$$G_1^{\mathcal{C}}[i_1] = \begin{bmatrix} G_1^{\mathcal{A}}[i_1] & G_1^{\mathcal{B}}[i_1] \end{bmatrix}, \quad G_d^{\mathcal{C}}[i_d] = \begin{bmatrix} G_d^{\mathcal{A}}[i_d] \\ G_d^{\mathcal{B}}[i_d] \end{bmatrix}.$$

При этом ТТ-ранги результата – сумма соответствующих ТТ-рангов слагаемых.

$$r_k^{\mathcal{C}} = r_k^{\mathcal{A}} + r_k^{\mathcal{B}}.$$

Поэлементное произведение тензоров.

Пусть тензоры $\mathcal{A}$, $\mathcal{B}$ заданы в ТТ-формате:

$$\mathcal{A}_{i_1 \dots i_d} = G_1^{\mathcal{A}}[i_1] \cdots G_d^{\mathcal{A}}[i_d], \quad \mathcal{B}_{i_1 \dots i_d} = G_1^{\mathcal{B}}[i_1] \cdots G_d^{\mathcal{B}}[i_d].$$

Тогда, чтобы найти ТТ-формат тензора

$$\mathcal{C} = \mathcal{A} \odot \mathcal{B}, \quad \mathcal{C}_{i_1 \dots i_d} = \mathcal{A}_{i_1 \dots i_d} \mathcal{B}_{i_1 \dots i_d},$$

достаточно построить его ТТ-ядра по формуле

$$G_k^{\mathcal{C}}[i_k] = G_k^{\mathcal{A}}[i_k] \otimes G_k^{\mathcal{B}}[i_k], \quad k = 1, \dots, d,$$

где $\otimes$ обозначает кронекерово произведение матриц.

При этом ТТ-ранги результата равны произведениям соответствующих ТТ-рангов сомножителей:

$$r_k^{\mathcal{C}} = r_k^{\mathcal{A}} r_k^{\mathcal{B}}.$$

Нетрудно заметить, что ранги ядер быстро растут, что существенно увеличивает количество хранимых данных. Хочется научиться находить тензор меньшего ранга, который является приближением заданного.

Скалярное произведение тензоров.

Пусть тензоры $\mathcal{A}$, $\mathcal{B}$ заданы в ТТ-формате:

$$\mathcal{A}_{i_1 \dots i_d} = G_1^{\mathcal{A}}[i_1] \cdots G_d^{\mathcal{A}}[i_d], \quad \mathcal{B}_{i_1 \dots i_d} = G_1^{\mathcal{B}}[i_1] \cdots G_d^{\mathcal{B}}[i_d].$$

Тогда скалярное произведение тензоров

$$\langle \mathcal{A}, \mathcal{B} \rangle = \sum_{i_1, \dots, i_d} \mathcal{A}_{i_1 \dots i_d} \mathcal{B}_{i_1 \dots i_d}$$

можно вычислить, не восстанавливая полные тензоры.

Для этого последовательно строятся матрицы:

$$Z_1 = \sum_{i_1=1}^{n_1} (G_1^{\mathcal{A}}[i_1])^\top G_1^{\mathcal{B}}[i_1],$$

$$Z_k = \sum_{i_k=1}^{n_k} (G_k^{\mathcal{A}}[i_k])^\top Z_{k-1} G_k^{\mathcal{B}}[i_k], \quad k = 2, \dots, d.$$

Тогда

$$\langle \mathcal{A}, \mathcal{B} \rangle = Z_d,$$

причем Z_d является матрицей размера 1×1 , то есть скаляром.

Тогда понятно, что норма Фробениуса тензора $\mathcal{A}$ в ТТ-формате вычисляется как

$$\|\mathcal{A}\|_F = \sqrt{\langle \mathcal{A}, \mathcal{A} \rangle}.$$

Умножение на скаляр.

Пусть тензор $\mathcal{A}$ задан в ТТ-формате:

$$\mathcal{A}_{i_1 \dots i_d} = G_1^{\mathcal{A}}[i_1] \cdots G_d^{\mathcal{A}}[i_d].$$

Тогда ТТ-формат тензора $\mathcal{B} = \lambda \mathcal{A}$ получается домножением любого одного ядра на λ , например первого:

$$G_1^{\mathcal{B}}[i_1] = \lambda G_1^{\mathcal{A}}[i_1], \quad G_k^{\mathcal{B}}[i_k] = G_k^{\mathcal{A}}[i_k], \quad k = 2, \dots, d.$$

ТТ-ранги не меняются.

Итак, посмотрим на алгоритм разложения тензора в ТТ-формат.

Алгоритм ТТ-SVD

Вход: Тензор $\mathcal{A} \in \mathbb{R}^{n_1 \times \dots \times n_d}$, относительная точность ε , ограничение ранга $R_{\max}$.

Выход: ТТ-ядра $G_1, \dots, G_d$.

1. Если $d = 1$, вернуть одно ядро $G_1 \leftarrow \text{reshape}(\mathcal{A}, [1, n_1, 1])$ и завершить работу.
2. Инициализация: $C \leftarrow \mathcal{A}$, $r_0 \leftarrow 1$.
3. Вычисляем локальный порог усечения δ :

$$\delta \leftarrow \begin{cases} \frac{\varepsilon}{\sqrt{d-1}} \|\mathcal{A}\|_F, & \text{если } \|\mathcal{A}\|_F > 10^{-30}, \\ 0, & \text{иначе.} \end{cases}$$

4. Для $k = 1$ до $d - 1$:

- **Развёртка:** Сформировать из тензора C матрицу $C_{(k)}$ размера $(r_{k-1}n_k) \times (n_{k+1} \dots n_d)$.
- **SVD:** Вычислить сингулярное разложение $C_{(k)} = U\Sigma V^\top$ с сингулярными числами $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.
- **Выбор ранга r_k :**
 - Определить числовой ранг $\hat{r}_k$ как максимальный индекс j , для которого $\sigma_j > \max(10^{-12}, 10^{-8}\sigma_1)$.
 - Положить $r_k \leftarrow \hat{r}_k$.
 - Если $\delta > 0$, последовательно уменьшать r_k , пока сумма квадратов отброшенных с конца сингулярных чисел не превысит порог: $\sum_{i=r_k+1}^{\hat{r}_k} \sigma_i^2 \leq \delta^2$.
 - Применить ограничение:

$$r_k \leftarrow \max(1, \min(r_k, R_{\max})).$$

- **Ядро:** Усечь U до первых r_k столбцов и свернуть в тензор:

$$G_k \leftarrow \text{reshape}(U_{:,1:r_k}, [r_{k-1}, n_k, r_k]).$$

- **Остаток:** Обновить C для следующего шага:

$$C \leftarrow \Sigma_{1:r_k, 1:r_k} V_{:, 1:r_k}^\top.$$

5. **Последнее ядро:** Свернуть матрицу-остаток C :

$$G_d \leftarrow \text{reshape}(C, [r_{d-1}, n_d, 1]).$$

Благодаря процедуре усечения, на каждом шаге k норма отброшенного хвоста строго контролируется локальным порогом:

$$\sum_{i > r_k} \sigma_i^2 \leq \delta^2.$$

Итоговая норма Фробениуса для ошибки аппроксимации $\hat{\mathcal{A}}$ ограничивается суммой ошибок всех шагов:

$$\|\mathcal{A} - \hat{\mathcal{A}}\|_F^2 \leq \sum_{k=1}^{d-1} \sum_{i > r_k} [\sigma_i^{(k)}]^2 \leq \sum_{k=1}^{d-1} \delta^2 = (d-1) \frac{\varepsilon^2 \|\mathcal{A}\|_F^2}{d-1}.$$

Таким образом, алгоритм гарантирует выполнение глобального условия $\|\mathcal{A} - \hat{\mathcal{A}}\|_F \leq \varepsilon \|\mathcal{A}\|_F$.

ТТ-представление не единственно: между соседними ядрами можно вставить произвольную обратимую матрицу M и её обратную:

$$G_k[i_k] G_{k+1}[i_{k+1}] = (G_k[i_k] M) (M^{-1} G_{k+1}[i_{k+1}]),$$

при этом тензор не изменится.

Этой свободой можно воспользоваться, чтобы привести ядра к ортогональной форме.

Правая ортогонализация ядра G_k

1. Развернём ядро $G_k \in \mathbb{R}^{r_{k-1} \times n_k \times r_k}$ в матрицу:

$$\hat{G}_k \in \mathbb{R}^{r_{k-1} \times (n_k r_k)}.$$

2. Выполним RQ-разложение (или QR матрицы $\hat{G}_k^\top$):

$$\hat{G}_k = RQ.$$

3. Обновим текущее ядро, свернув матрицу Q обратно в тензор:

$$G_k \leftarrow \text{reshape}(Q, [r_{k-1}, n_k, r_k]).$$

Ядро G_k теперь называется *правоортогональным*.

4. Поглотим матрицу R в предыдущее ядро G_{k-1} . Для каждого индекса $i_{k-1} = 1, \dots, n_{k-1}$ выполним матричное умножение:

$$G_{k-1}[i_{k-1}] \leftarrow G_{k-1}[i_{k-1}] R.$$

NB: *полной правой ортогонализацией* называется последовательное выполнение правой ортогонализации для k от d до 2. Говорят, что тензор находится в право-канонической форме.

Полная правая ортогонализация позволяет перенести всю информацию о норме тензора в первое ядро G_1 , сделав остальные ядра ортогональными.

Левая ортогонализация ядра G_k

1. Развернём ядро $G_k \in \mathbb{R}^{r_{k-1} \times n_k \times r_k}$ в матрицу:

$$\hat{G}_k \in \mathbb{R}^{(r_{k-1} n_k) \times r_k}.$$

2. Выполним QR-разложение:

$$\hat{G}_k = QR.$$

3. Обновим текущее ядро, свернув матрицу Q обратно в тензор:

$$G_k \leftarrow \text{reshape}(Q, [r_{k-1}, n_k, r_k]).$$

Ядро G_k теперь называется *левоортогональным*.

4. Поглотим матрицу R в следующее ядро G_{k+1} . Для каждого индекса $i_{k+1} = 1, \dots, n_{k+1}$ выполним матричное умножение:

$$G_{k+1}[i_{k+1}] \leftarrow R G_{k+1}[i_{k+1}].$$

NB: *полной левой ортогонализацией* называется последовательное выполнение левой ортогонализации для k от 1 до $d - 1$. Говорят, что тензор находится в лево-канонической форме.

Полная левая ортогонализация позволяет перенести всю информацию о норме тензора в последнее ядро G_d , сделав остальные ядра ортогональными.

Рассмотрим процедуру замены ТТ-тензора $\mathcal{A}$ на близкий ТТ-тензор $\tilde{\mathcal{A}}$ меньших рангов, такой что

$$\|\mathcal{A} - \tilde{\mathcal{A}}\|_F \leq \varepsilon \|\mathcal{A}\|_F.$$

Это критически важно, поскольку после базовых арифметических операций ТТ-ранги тензоров неизбежно возрастают.

ТТ-округление (ТТ-rounding).

1. Правый проход (полная правая ортогонализация).
2. Левый проход^a (SVD-усечение).

Осуществляется проход слева направо ($k = 1, \dots, d - 1$).

Зададим локальный порог ошибки:

$$\delta = \frac{\varepsilon \|G_1\|_F}{\sqrt{d-1}}.$$

Для каждого ядра G_k :

- (a) Разворачиваем G_k в матрицу $\hat{G}_k \in \mathbb{R}^{(r_{k-1}n_k) \times r_k}$.
- (b) Вычисляем сингулярное разложение: $\hat{G}_k = U\Sigma V^\top$.
- (c) **Выбор нового ранга $\tilde{r}$:** находим минимальное значение $\tilde{r} \geq 1$, при котором сумма квадратов *отброшенных* сингулярных чисел не превышает δ^2 :

$$\sum_{i=\tilde{r}+1}^R \sigma_i^2 \leq \delta^2.$$

- (d) Сворачиваем левые сингулярные векторы в новое сжатое ядро $\tilde{G}_k$:

$$\tilde{G}_k \leftarrow \text{reshape}(U_{[:, 1:\tilde{r}]}, [r_{k-1}, n_k, \tilde{r}]).$$

- (e) Поглощаем остаток в следующее ядро G_{k+1} :

$$G_{k+1}[i_{k+1}] \leftarrow \left(\Sigma_{[1:\tilde{r}, 1:\tilde{r}]} V_{[:, 1:\tilde{r}]}^\top \right) G_{k+1}[i_{k+1}].$$

^aЭтот этап алгоритмически идентичен полной левой ортогонализации, но вместо QR-разложения матриц $\hat{G}_k$ применяется SVD с отсечением малых сингулярных чисел.

В результате получается тензор $\tilde{\mathcal{A}}$ с уменьшенными ТТ-рангами и гарантированной относительной ошибкой не более ε .

ПРАКТИЧЕСКАЯ ЧАСТЬ

В рамках практической части лабораторной работы вам предстоит реализовать работу с тензорами в плотном формате, TT-SVD разложение и работу с тензорами в TT-формате.

Репозиторий с кодом, который вам предстоит дополнить, имеет следующую структуру:

```
├── algorithms
│   ├── canonical_form.py
│   ├── tensor_operations.py
│   ├── tt_round.py
│   └── tt_svd.py
├── core
│   ├── dense_tensor.py
│   ├── linalg.py
│   ├── tt_tensor.py
│   └── utils.py
├── processor_type
│   ├── cpu_operations.py
│   └── interface.py
```

При этом обратите внимание, что `linalg.py`, `cpu_operations.py`, `interface.py` уже заполнены. Ниже приведено краткое описание репозитория:

- **algorithms**
 - `canonical_form.py` – полная правая и левая ортогонализация TT-тензора;
 - `tensor_operations.py` – базовые операции над TT-тензорами без восстановления полного тензора;
 - `tt_round.py` – алгоритм TT-округления;
 - `tt_svd.py` – алгоритм TT-SVD для декомпозиции тензора в TT-формат.
- **core**
 - `linalg.py` – готовая реализация базовых матричных алгоритмов, используемых в TT-SVD;
 - `tt_tensor.py` – класс `TTTensor`, описывающий структуру данных TT-тензора и базовые операции;

- `dense_tensor.py` – класс `DenseTensor`, реализующий стандартный многомерный массив и базовые преобразования;
- `utils.py` – вспомогательные и отладочные функции;
- `processor_type`
 - `cpu_operations.py` – реализация вычислительного бэкенда для работы на CPU;
 - `interface.py` – абстрактный базовый класс `BackendInterface`, определяющий единый контракт для всех вычислительных бэкендов.

Рекомендуется заполнять репозиторий в следующем порядке:

1. `utils.py`
2. `dense_tensor.py`
3. `tt_tensor.py`
4. `algorithms`

Следует придерживаться алгоритмов, описанных в теоретической части, чтобы при прохождении тестов минимизировать вероятность ошибки из-за неединственности разложений. Например, после выполнения TT-SVD должен получиться TT-тензор в полной лево-ортогональной форме, это условие будет проверяться.

Для выполнения работы требуется репозиторий: https://github.com/RainDmitriy/la_sem2_lab01_aie

Перед началом работы необходимо создать свою копию репозитория:

1. Сделать **Fork** репозитория для создания своего репозитория.
2. Клонировать свою копию локально:

```
git clone https://github.com/USERNAME/la_sem2_lab01_aie.git
cd la_sem2_lab01_aie
```

3. Добавить основной template-репозиторий как дополнительный remote, чтобы можно было создавать Pull Request:

```
git remote add upstream https://github.com/RainDmitriy/la_sem2_lab01_aie.git
git fetch upstream
```

4. Создать ветку для решения задания и работать в ней:

```
git checkout -b my_solution
```

5. Заполнить пропущенные реализации.
6. После выполнения задания коммитить и пушить изменения в свой репозиторий:

```
git add .
git commit -m "Homework solution"
git push origin my_solution
```

7. Создать Pull Request в template repo (upstream) для запуска автотестов через веб-интерфейс.
8. Дождаться запуска Workflow и увидеть результат тестирования, если тесты прошли, всё хорошо.

ПОСЛЕСЛОВИЕ

“I am part of that power which eternally
wills evil and eternally works good.”

Вот и подошло наше путешествие по миру линейной алгебры к концу. Надеемся, что мы смогли дать не только конкретные знания, но и метод для самостоятельного изучения любого вопроса, который бы ни возник на вашем пути.

Спасибо!

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. И. Р. Шафаревич, Основные понятия алгебры, Наука, Москва, 1999.
2. I. V. Oseledets, Tensor-Train Decomposition, SIAM Journal on Scientific Computing, 33 (2011), pp. 2295–2317.
3. Лекция "Разложение в тензорный поезд в задачах машинного обучения", Александр Новиков - ФКН ВШЭ/ИВМ РАН.
4. Лекция "Малоранговая аппроксимация тензоров", Рахуба М.В. - ФПМИ.
5. Tamara G. Kolda, Brett W. Bader, Tensor Decompositions and Applications, SIAM Review
6. Tensor-Train SVD (TT-SVD) Algorithm